

Module 6 Question Pool

OMSCS 6250 Computer Networks

Lesson 6: Router Design and Algorithms (Part 2)

Why We Need Packet Classification?

Q1. [MCQ]

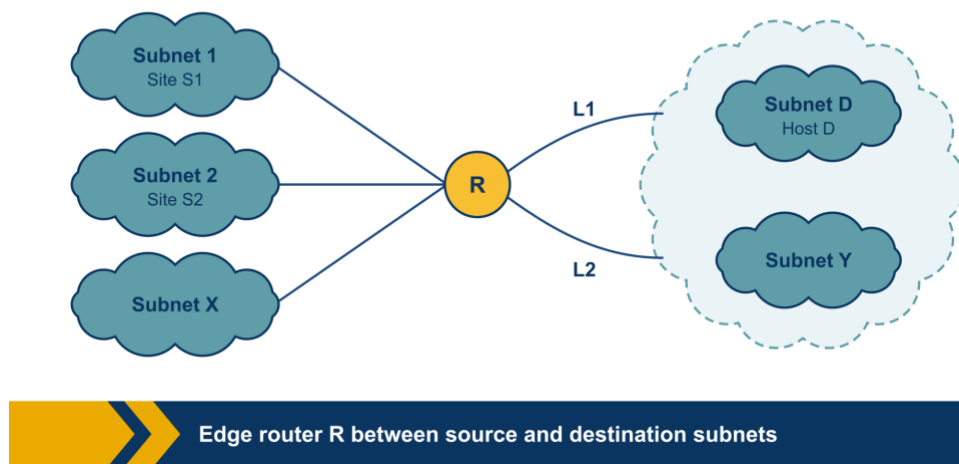


Figure: Edge router R between source and destination subnets (Module 6)

Consider edge router R, which connects three subnets to two outbound links, L1 and L2. Site S1 is in Subnet 1, site S2 is in Subnet 2, and the third subnet is Subnet X. Links L1 and L2 lead to remote networks, including Subnet D, which holds host D, and Subnet Y.

Router R forwards traffic by longest prefix matching on the destination IP address. Router R must also apply three rules:

Rule 1: send video traffic from site S1 to host D over link L1, and all other traffic to D over link L2.

Rule 2: drop all traffic from site S2.

Rule 3: reserve 50 Mbps for traffic from Subnet X to Subnet Y.

Which of the three rules can router R apply using longest prefix matching on the destination IP address alone?

- A. Rule 2 only. A table entry for Subnet 2 matches the packets that site S2 sends.
- B. Rules 1 and 3 only. An entry for Subnet D or Subnet Y selects the traffic each rule covers.
- C. All three. The operator can add a more specific destination prefix to match each rule.
- D. None. The lookup returns the next hop per prefix, regardless of the source or traffic type.

Correct answer: **D**

Why: Longest prefix matching reads only the destination address. Every packet to one prefix gets the same next hop, however specific the prefix is. Each rule also depends on the source, and Rule 1 depends on the traffic type as well. An entry for Subnet 2 matches packets sent to Subnet 2, not packets sent from site S2. Matching several header fields at once is packet classification.

Packet Classification: Simple Solutions

Q2. [TF]

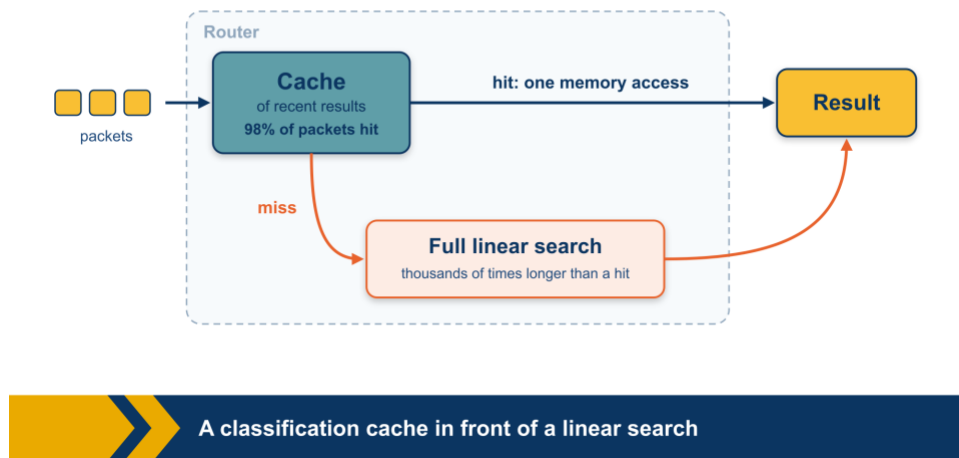


Figure: A classification cache in front of a linear search (Module 6)

Consider a router that classifies packets by a linear search through a large list of rules. In front of the search, the router keeps a cache of recent results. On a cache hit, the router finds the result in one memory access. On a cache miss, the router runs the full linear search, which takes thousands of times longer than a cache hit. The cache hit rate is 98 percent.

Most of the average classification time comes from the cache misses.

- True
- False

Correct answer: **True**

Why: A cache miss still costs a full linear search. Only 2 percent of packets miss, but each miss takes thousands of times as long as a hit. The misses add far more to the average than the hits do. A high hit rate cannot hide a slow miss.

Fast Searching Using Set-Pruning Tries

Q3. [MCQ]

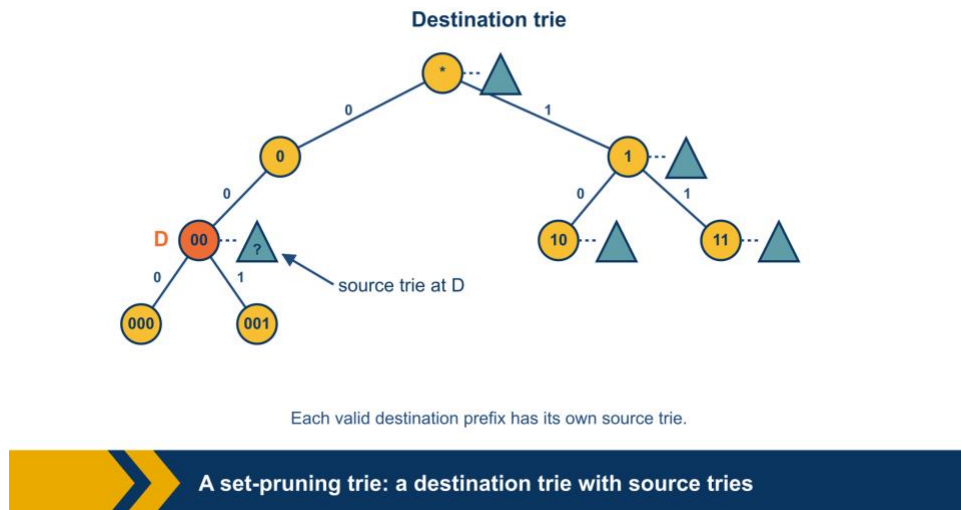


Figure: A set-pruning trie: a destination trie with source tries (Module 6)

Consider a two-field packet classifier that matches packets on a destination prefix and a source prefix. The classifier is a set-pruning trie. A destination trie indexes the destination prefixes, and each valid destination node points to its own source trie.

Let D be one valid destination node, such as 00^* .

Which rules must the source trie at node D hold?

- **A.** Only the rules whose destination prefix is exactly D , stored once in that source trie.
- **B.** Every rule whose destination prefix is D or a longer prefix below D , such as 000^* or 001^* .
- **C.** Every rule whose destination prefix is D or a shorter prefix above D , such as * .
- **D.** Only the rules that remain after broader rules overlapping with D 's rules are pruned out.

Correct answer: **C**

Why: A packet that matches destination prefix D also matches every shorter prefix of D , up to the wildcard * . Rules with those destinations can apply to it, so the source trie at D holds copies of them all. One walk down that trie finds the least-cost rule. Keeping only the rules for D is the backtracking layout. A longer prefix such as 000^* need not match the packet.

Q4. [MCQ]

Consider a router that performs two-field packet classification on N rules with a set-pruning trie. Some rules have the wildcard destination prefix * .

As N grows, which cost of the set-pruning trie grows fastest in the worst case?

- **A.** Lookup time, because each search climbs back up to check the source tries of ancestors.
- **B.** Update time, because the switch pointers must be recomputed every time a rule changes.
- **C.** Power use, because the router compares every stored rule in parallel on each lookup.
- **D.** Memory, because rules with broad destination prefixes are copied into many source tries.

Correct answer: **D**

Why: A rule with the wildcard destination * matches every destination. Set pruning copies such a rule into the source trie of every destination prefix, so memory can grow to about N squared. A lookup is still one walk down the destination trie and one source trie. Climbing to ancestor tries is the cost of backtracking, and recomputing switch pointers is the cost of the grid of tries. Comparing every rule in parallel is how a ternary CAM works.

Q5. [MCQ]

Rule	Destination prefix	Source prefix
R1	1*	00*
R2	10*	1*
R3	*	01*
R4	101*	0*
R5	11*	10*
R6	10*	11*



Figure: A rule database with six two-field rules (Module 6)

Consider a router that classifies packets on two fields, the destination prefix and the source prefix. The rule database of the router contains the six rules in the table. The router stores these rules in a set-pruning trie. Each distinct destination prefix in the table has its own source trie.

In how many source tries does the set-pruning trie store rule R3?

- A. One: only the source trie at its own destination prefix, *.
- B. Four: the source tries of the other destination prefixes in the table.
- C. Five: the source tries of all the destination prefixes in the table.
- D. Six: one source trie for each of the six rules in the table.

Correct answer: C

Why: The destination prefix * matches every destination, so rule R3 can apply to any packet. Set pruning copies R3 into the source trie of every destination prefix in the table: *, 1*, 10*, 11* and 101*. That makes five tries. The trie at * counts, because a packet whose destination starts with 0 ends its destination search there. R2 and R6 share 10*, so six rules give five tries. Keeping R3 only at * is the backtracking layout.

Reducing Memory Using Backtracking

Q6. [MCQ]

Consider a router that performs two-field packet classification on destination and source prefixes. The router holds N rules, and each field is at most W bits long. The router replaces its set-pruning trie with a backtracking trie.

What does the router give up?

- A. Lookup speed.
- B. Accuracy.
- C. Update speed.
- D. Memory.

Correct answer: **A**

Why: Backtracking stores each rule once, in the source trie of its own destination prefix. Memory falls from $O(N^2)$ to $O(NW)$. A matching rule may sit at an ancestor destination prefix, so the search climbs back up and checks up to W source tries. Worst-case lookup rises to $O(W^2)$. No rule is missed. Switch pointers belong to the grid of tries.

Q7. [MCQ]

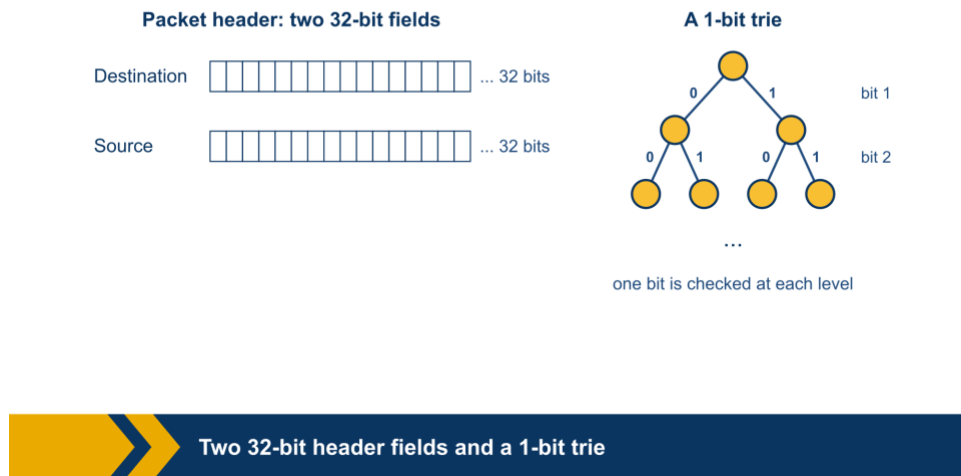


Figure: Two 32-bit header fields and a 1-bit trie (Module 6)

Consider a router that classifies packets on two header fields: the destination prefix and the source prefix. Each field is 32 bits long. The classifier is designed as a backtracking trie built from 1-bit tries.

In the worst case, about how many memory accesses does it take to classify a single packet?

- A. 64 accesses, 32 in the destination trie plus 32 in a single source trie.
- B. 32 accesses, one for each of the 32 header bits in a single walk.
- C. 1,024 accesses, because up to 32 source tries are searched at 32 each.
- D. 8 accesses, one for each 4-bit step through a 32-bit field.

Correct answer: **C**

Why: W is the number of bits in each field, so $W = 32$ here. With 1-bit tries, one walk down a trie takes up to W accesses. Backtracking walks the destination trie, then searches the source trie of the longest match and of every ancestor with a nonempty

source trie. In the worst case, that is W source tries at W accesses each, or $32 \times 32 = 1,024$. A count of 32 is a single walk, and 64 is two walks. A count of 8 is a single walk with 4-bit tries.

Grid of Tries

Q8. [MCQ]

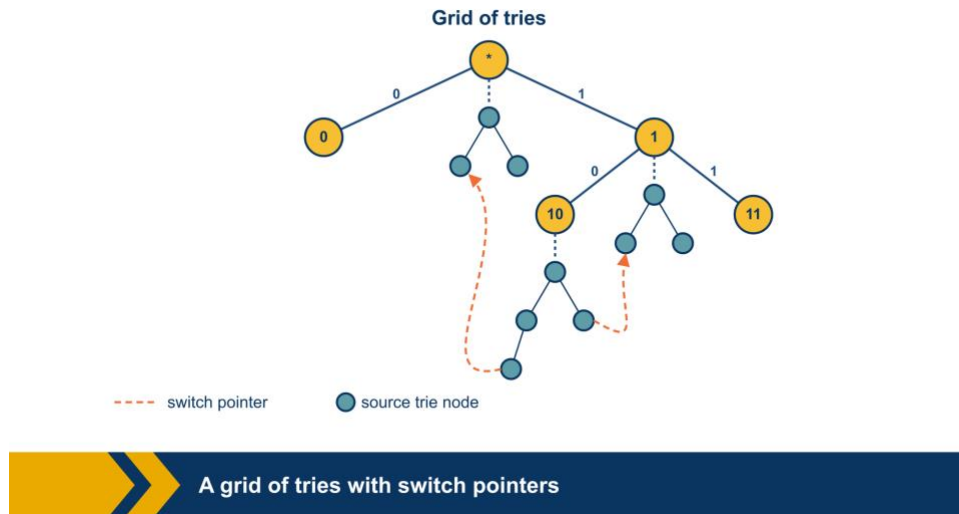


Figure: A grid of tries with switch pointers (Module 6)

Consider a router that classifies packets on two header fields. The router replaces its backtracking trie with a grid of tries.

The grid of tries stores a precomputed switch pointer at each failure point in the source tries.

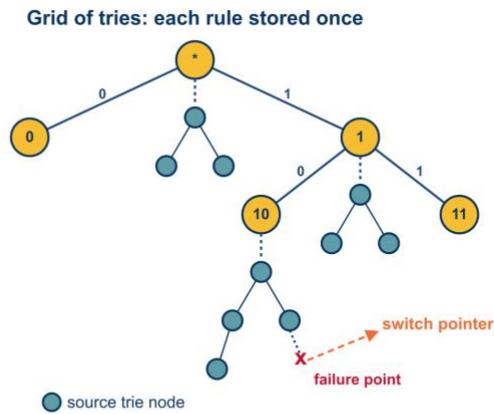
What does the router give up, compared with backtracking?

- A. Memory.
- B. Update speed.
- C. Accuracy.
- D. Worst-case lookup speed.

Correct answer: **B**

Why: Switch pointers are computed from the rule set, so a rule change means recomputing them. Updates are far rarer than lookups, so the trade is acceptable. Each rule is still stored once, so memory stays $O(NW)$. The pointers replace the climb back up the destination trie, so the worst-case lookup falls to $O(W)$. The tries they skip hold only source prefixes shorter than the best match already found.

Q9. [MCQ]



A grid of tries with a switch pointer

Figure: A grid of tries with a switch pointer (Module 6)

Consider a grid of tries for two-field packet classification. Each rule is stored exactly once in the source trie of its own destination prefix node. At every failure point in a source trie, the structure embeds a precomputed switch pointer.

Which step of the backtracking search do the switch pointers replace?

- **A.** Traversing the source trie associated with the longest matching destination prefix.
- **B.** Tracking and updating the best matching rule found so far during the search.
- **C.** Climbing back up the destination trie to search the source tries of ancestor nodes.
- **D.** Re-running the destination prefix lookup, because the pointer caches its earlier result.

Correct answer: **C**

Why: When a source trie search fails, plain backtracking climbs back up the destination trie to search the source tries of ancestors. A precomputed switch pointer at the failure point jumps straight to the next source trie that could still hold a match. The search still walks the source trie at the longest destination match, and it still tracks the best match so far. The destination lookup runs once.

Scheduling and Head of Line Blocking

Q10. [MCQ]

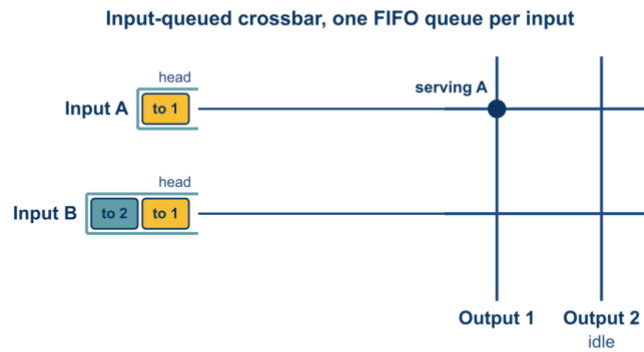


Figure: An input-queued crossbar with one FIFO queue per input (Module 6)

Consider an input-queued crossbar switch where each input port maintains a single FIFO queue. The crossbar is scheduled using the take a ticket scheme.

During the current packet time slot:

The head-of-line packet at Input A is destined for Output 1.

The head-of-line packet at Input B is also destined for Output 1.

The scheduler awards Output 1 to Input A.

The second packet in queue at Input B is destined for Output 2, which is idle.

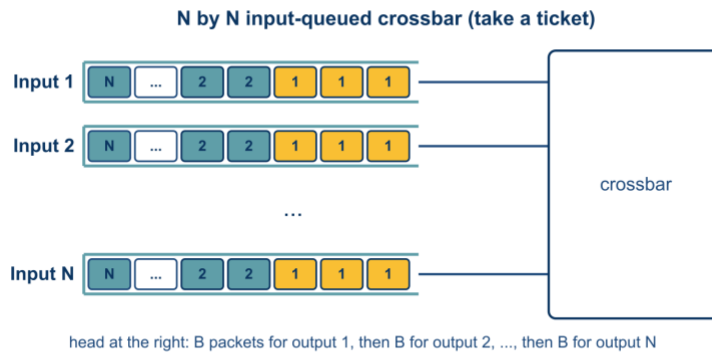
What packet does Input B transmit during this time slot?

- **A.** Nothing, because its packet for Output 2 waits behind the head-of-line packet for Output 1.
- **B.** Its second packet (destined for Output 2), which bypasses the blocked head-of-line packet.
- **C.** Its head-of-line packet, which Output 1 carries at the same time as Input A's packet.
- **D.** Its head-of-line packet, redirected to idle Output 2 so that Output 2 does not sit idle.

Correct answer: A

Why: Input B has a single FIFO queue, so only its head-of-line packet can be considered. Output 1 is granted to Input A, so that packet waits. The packet for Output 2 waits behind it, although Output 2 is idle. This wait is head-of-line blocking. A crossbar output connects to one input at a time, and a packet goes only to the output its lookup chose.

Q11. [TF]



➤ The FIFO input queues of a crossbar router

Figure: The FIFO input queues of a crossbar router (Module 6)

Consider router R, which has an N by N input-queued crossbar switch. The switch is scheduled by the take a ticket algorithm. Each input port uses a single FIFO queue.

Every input queue holds a large number B of packets for Output 1, then B packets for Output 2, and so on up to Output N.

While the switch works through the packets for Output 1, several inputs can send packets at the same time.

- True
- False

Correct answer: **False**

Why: While the switch works through the packets for Output 1, every input has a packet for Output 1 at the head of its queue. Output 1 can take only one packet per time slot, so only one input sends and the other N-1 wait. The other N-1 outputs sit idle. For large B, throughput falls to 1/N of capacity.

Avoiding Head of Line Blocking

Q12. [MCQ]

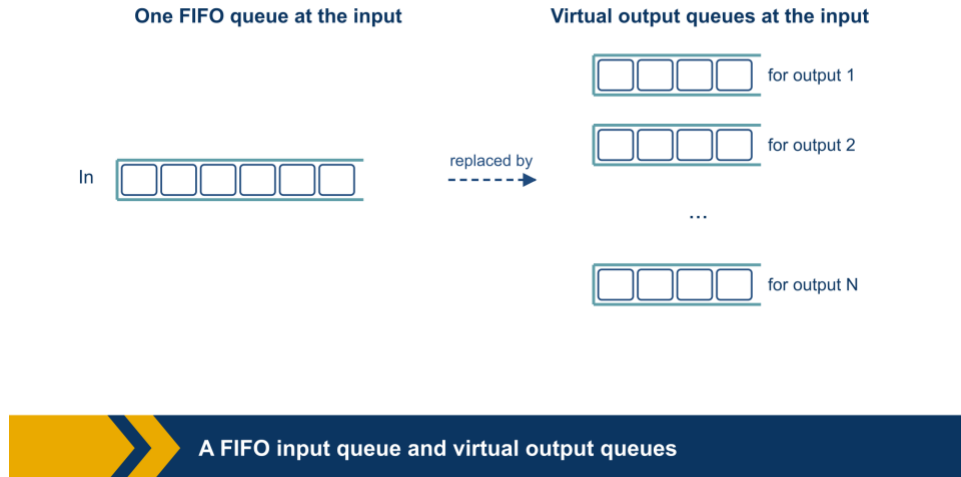


Figure: A FIFO input queue and virtual output queues (Module 6)

Consider an N by N crossbar switch with input queueing. The designer replaces the single FIFO queue at each input port with N Virtual Output Queues (VOQs), one per output port.

Why does this change remove head-of-line blocking?

- **A.** Each input port can now transmit packets from all N of its VOQs in a single time slot.
- **B.** Packets for an idle output port no longer share a queue with packets for a busy output port.
- **C.** Packets for busy output ports are dropped from their VOQs, so they no longer block others.
- **D.** Concentrator hardware randomly selects winning packets whenever inputs contend for an output.

Correct answer: **B**

Why: A single FIFO queue causes head-of-line blocking because packets for different outputs share one line. VOQs split the packets at each input port into N queues, one per output. A packet for an idle output can then be scheduled while a packet for a busy output waits in its own VOQ. Each input still sends from one VOQ per time slot. Concentrators belong to Knockout, which removes input queueing instead of splitting it.

Q13. [MCQ]

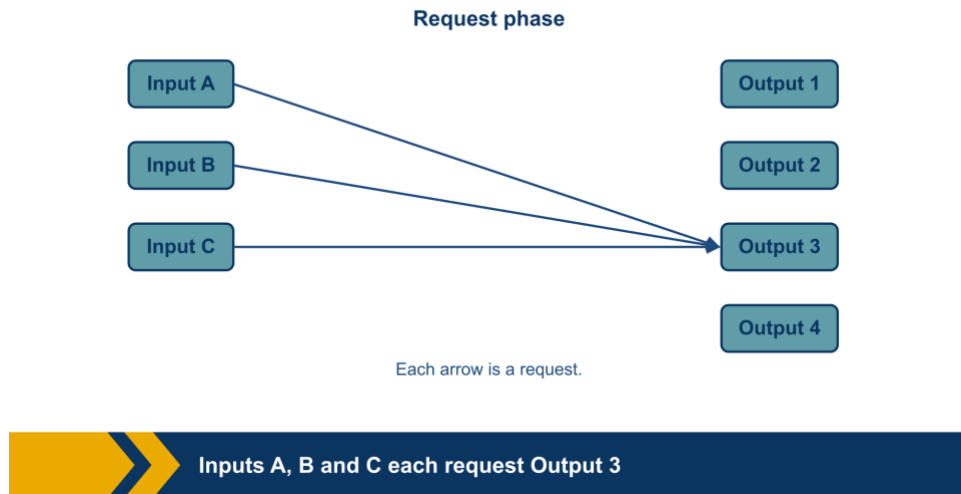


Figure: Inputs A, B and C each request Output 3 (Module 6)

Consider an input-queued crossbar switch using Virtual Output Queues (VOQs), scheduled by Parallel Iterative Matching (PIM).

In the Request phase of a round, Inputs A, B and C all send requests to Output 3.

How does Output 3 respond in the Grant phase?

- **A.** It randomly selects one input among A, B, and C, and grants only that input.
- **B.** It queues all three requests and grants them in arrival order over later rounds.
- **C.** It grants the input that sent the fewest requests, to maximize the total matches.
- **D.** It grants requests to all three inputs at once, letting the inputs decide which one sends.

Correct answer: **A**

Why: PIM runs three phases in each round: Request, Grant and Accept. In the Grant phase, an output that receives requests from several inputs picks exactly one input at random and grants that input. The choice is random, so it does not depend on arrival order or on how many requests each input sent. The inputs choose among their grants only in the Accept phase.

Q14. [MCQ]

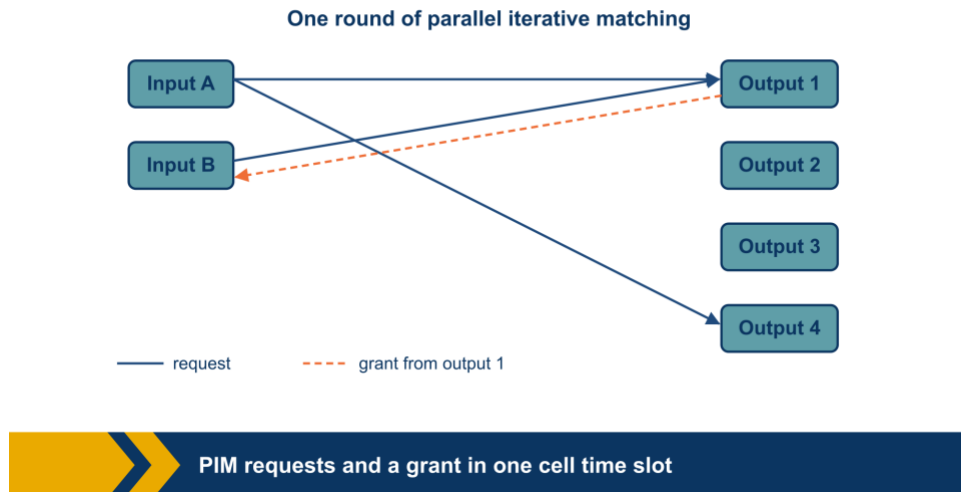


Figure: PIM requests and a grant in one cell time slot (Module 6)

Consider a crossbar switch with Virtual Output Queues (VOQs) and scheduled by Parallel Iterative Matching (PIM).

In the Request phase of the current cell time slot, Input A requests Output 1 and Output 4, and Input B requests Output 1 only. No other input requests Output 4. In the Grant phase, Output 1 picks Input B at random and grants it.

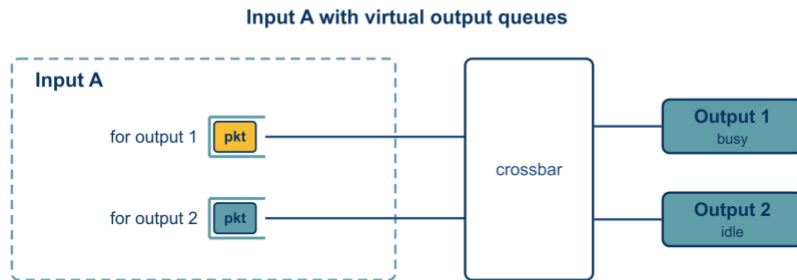
What happens to Input A in the rest of this cell time slot?

- **A.** Input A transmits nothing during this cell time slot and retries Output 1 in the next slot.
- **B.** Output 4 grants Input A, so Input A accepts it and sends to Output 4 in this cell time slot.
- **C.** Input A transmits to Output 1 immediately after Input B finishes, within the same cell time slot.
- **D.** Output 4 waits for Output 1 to decide, so Input A receives its grant in the next cell time slot.

Correct answer: **B**

Why: In PIM, each input requests every output it has queued packets for, and each output grants on its own. Output 4 has only Input A's request, so it grants Input A. Input A holds one grant and accepts it. Input A sends to Output 4 while Input B sends to Output 1. An output connects to one input per cell time slot, so Input A cannot follow Input B on Output 1.

Q15. [TF]



➤ Virtual output queues (VOQs) at Input A of a crossbar

Figure: Virtual output queues (VOQs) at Input A of a crossbar (Module 6)

Consider an input-queued crossbar switch using Virtual Output Queues (VOQs) and scheduled by Parallel Iterative Matching (PIM).

Input A holds packets in its VOQs for busy Output 1 and for idle Output 2.

In the Request phase of the current round, Input A requests Output 2 without waiting for its packet for Output 1 to be sent.

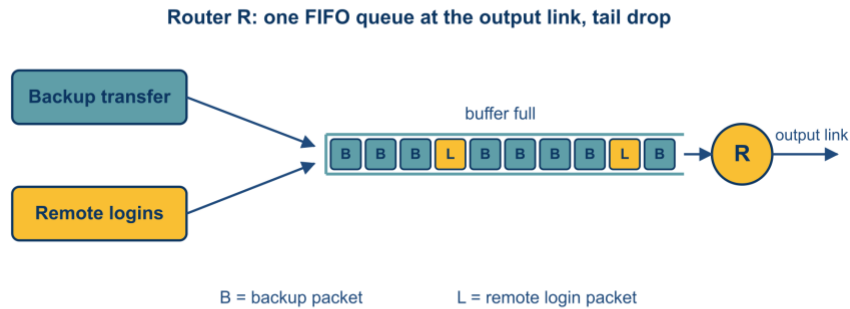
- True
- False

Correct answer: **True**

Why: Virtual output queues keep a separate queue for each output port. In the Request phase of PIM, an input sends requests in parallel to all output ports for which it has queued packets. Input A requests Output 1 and Output 2 at the same time.

Scheduling Introduction

Q16. [MCQ]



➤ A FIFO output queue with tail drop at a router

Figure: A FIFO output queue with tail drop at a router (Module 6)

Consider an output link queue on router R. The queue is served First-In, First-Out (FIFO) and uses tail drop: when the buffer is full, each newly arriving packet is dropped.

A bulk backup transfer and several interactive remote login sessions share the output link. Traffic rises until the buffer is full.

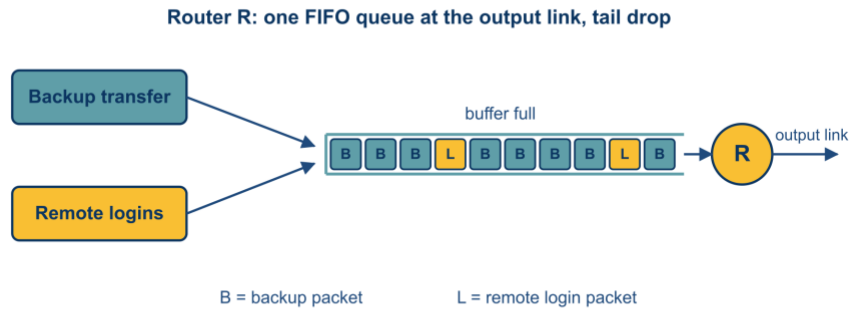
What is the drawback of FIFO with tail drop in this situation?

- **A.** Tail drop discards every packet that reaches the full buffer, so the backup locks out the sessions.
- **B.** Tail drop selectively discards bulk backup packets first, so the backup flow gets less than its fair share.
- **C.** Scheduling overhead is too high for gigabit links, so packets wait while the scheduler decides.
- **D.** Tail drop reorders packets within individual flows, which TCP then reads as congestion signals.

Correct answer: **A**

Why: Tail drop discards any packet that arrives while the buffer is full, from any flow. The bulk transfer fills the whole buffer. Packets from the remote login sessions then arrive at a full buffer and are dropped. The interactive sessions are locked out. FIFO decisions are fast enough for links above 40 gigabit, and one FIFO queue keeps each flow's packets in order.

Q17. [MCQ]



➤ A FIFO output queue with tail drop at a router

Figure: A FIFO output queue with tail drop at a router (Module 6)

Consider an output link queue on router R. The queue is served First-In, First-Out (FIFO) and uses tail drop: when the buffer is full, each newly arriving packet is dropped.

A bulk backup transfer and several remote login sessions share the output link. The backup keeps the buffer full, so packets from the remote login sessions are dropped as they arrive.

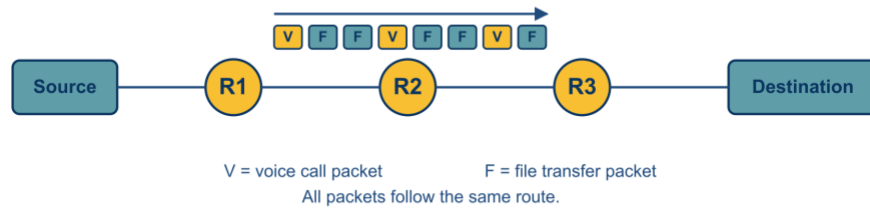
What does router R need so that the backup can no longer lock out the remote login sessions?

- **A.** A larger buffer, so that remote login packets find free space when they arrive.
- **B.** A faster FIFO scheduler, so that packets leave before the buffer can fill.
- **C.** A single queue at the input port instead, so packets wait before they cross the switch.
- **D.** A way to tell the flows apart, so it can give each flow a fair share of the link.

Correct answer: **D**

Why: A single FIFO queue with tail drop has no say in which packets it drops, so the backup takes whatever buffer there is. A larger buffer fills the same way. FIFO is already fast enough for links above 40 gigabit, so speed is not the problem. One queue at the input is still one shared queue, and the lockout stays. The router must recognize each flow from its header fields, which is packet classification, so that a scheduler can give each flow its share.

Q18. [MCQ]



A stream of voice and file transfer packets

Figure: A stream of voice and file transfer packets (Module 6)

Consider a packet stream that travels a fixed route from a source to a destination. The stream carries one voice call and one file transfer, which can be told apart by their header fields:

Voice call: needs strict low-latency guarantees at every router.

File transfer: needs high throughput, with no strict delay bounds.

How many flows is the packet stream considered to contain?

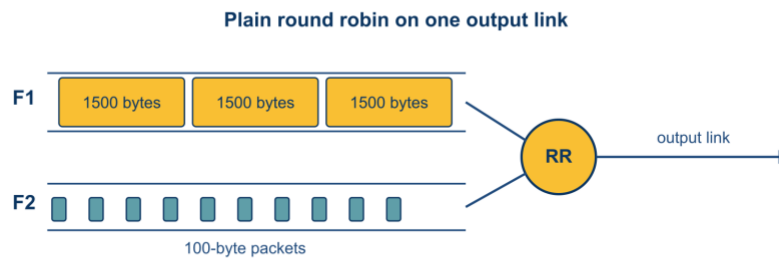
- A. One flow, because all of its packets travel the same route from source to destination.
- B. One flow, because all of its packets can be identified from the same header fields.
- C. Two flows, because the two traffic types need different grades of service.
- D. Two flows, because voice packets are smaller than file transfer packets.

Correct answer: C

Why: A flow shares one route and one grade of service. Voice needs strict delay bounds and file transfer does not, so there are two flows. A shared route or shared header fields is not enough. Packet size plays no part.

Deficit Round Robin

Q19. [MCQ]



Flows F1 and F2 under plain round robin

Figure: Flows F1 and F2 under plain round robin (Module 6)

Consider two flows, F1 and F2, that share a link under plain round robin. Both flows always have packets queued. F1 sends 1,500-byte packets, and F2 sends 100-byte packets.

The router then switches to Deficit Round Robin (DRR), which gives each flow a quantum and a deficit counter.

Which fairness problem of plain round robin does DRR correct?

- **A.** Plain round robin grants F2 15 times more bandwidth, because smaller packets finish sooner.
- **B.** Plain round robin gives no delay bound, which DRR then guarantees with its deficit counter.
- **C.** Plain round robin gives both flows equal bandwidth, although F1's packets are 15 times larger.
- **D.** Plain round robin grants F1 15 times more bandwidth, as it counts packets rather than bits.

*Correct answer: **D***

Why: Plain round robin sends one packet per flow in each round. F1's packets are 15 times larger than F2's, so F1 gets 15 times more bandwidth than F2. DRR corrects this by giving each flow a quantum in bytes and tracking its deficit across rounds. DRR guarantees bandwidth and gives no delay guarantee.

Q20. [MCQ]

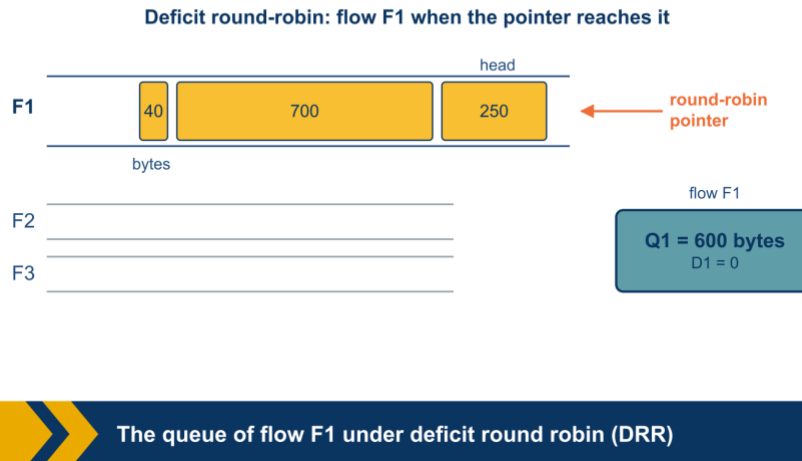


Figure: The queue of flow F1 under deficit round robin (DRR) (Module 6)

Consider an output port scheduled by Deficit Round Robin (DRR). Flow F1 has a quantum $Q1 = 600$ bytes, and its deficit counter $D1$ starts at 0.

F1's queue holds three packets, in order: 250 bytes, 700 bytes and 40 bytes.

What does the scheduler do when it visits F1 in this round?

- **A.** F1 transmits the 250-byte packet and retains a deficit counter of $D1 = 350$ bytes for its next turn.
- **B.** F1 transmits the 250-byte packet, and its deficit counter of 350 bytes is reset to 0 at the end of the turn.
- **C.** F1 transmits both the 250-byte and 700-byte packets, leaving its deficit counter at -350 bytes.
- **D.** F1 transmits the 250-byte and 40-byte packets by skipping the 700-byte packet, keeping a deficit of 310 bytes.

Correct answer: **A**

Why: The scheduler adds $Q1$ to $D1$, so F1 has $0 + 600 = 600$ bytes of credit. F1 sends the 250-byte head packet, leaving $600 - 250 = 350$ bytes. The 700-byte second packet exceeds 350 bytes, so F1's turn ends. F1's queue is not empty, so $D1 = 350$ carries over to the next round. The deficit counter never goes below zero. DRR also never serves packets out of queue order.

Traffic Scheduling: Token Bucket

Q21. [MCQ]

Consider a sender whose traffic is covered by a Service Level Agreement (SLA). The SLA sets an agreed traffic rate. The sender bursts above this rate. An edge router can enforce the SLA by traffic policing or by traffic shaping.

How does traffic policing differ from traffic shaping?

- **A.** Policing queues non-conforming packets until they conform; shaping drops or marks them at once.
- **B.** Policing drops or marks non-conforming packets at once; shaping queues them until they conform.

- C. Policing checks the average rate once per second; shaping checks each packet as it arrives.
- D. Policing enforces the long-run average rate; shaping limits the largest burst size.

Correct answer: **B**

Why: Policing checks each packet against the SLA rate and drops or marks non-conforming packets immediately. Shaping holds excess packets in per-flow queues and releases them at a conforming rate. Both enforce the same agreement; they differ in what happens to the excess.

Q22. [MCQ]

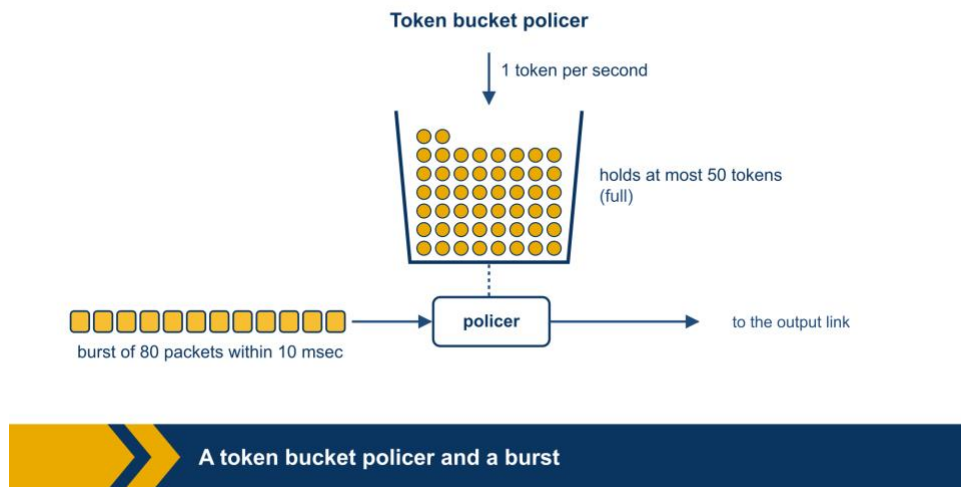


Figure: A token bucket policer and a burst (Module 6)

Consider a router that polices a flow with a token bucket. Tokens arrive at 1 token per second, and the bucket holds at most 50 tokens. Each packet needs one token.

The flow has been idle, so the bucket is full. Then a burst of 80 packets arrives within 10 milliseconds, too fast for any new token to arrive.

How does the policer handle the 80 packets?

- A. It passes all 80 packets immediately, and the token count drops to -30.
- B. It passes 1 packet per second, the token rate, and drops or marks the remaining 79 packets.
- C. It passes the first 50 packets immediately, and drops or marks the remaining 30 packets.
- D. It passes the first 50 packets immediately, and buffers the remaining 30 until new tokens arrive.

Correct answer: **C**

Why: The full bucket holds 50 tokens, so the first 50 packets each take a token and pass immediately. No new tokens arrive during the 10 ms burst, and the token count never goes below zero. The remaining 30 packets find the bucket empty. A policer drops or marks them immediately and does not buffer them. The saved tokens let a quiet flow send a burst above the token rate.

Q23. [MCQ]

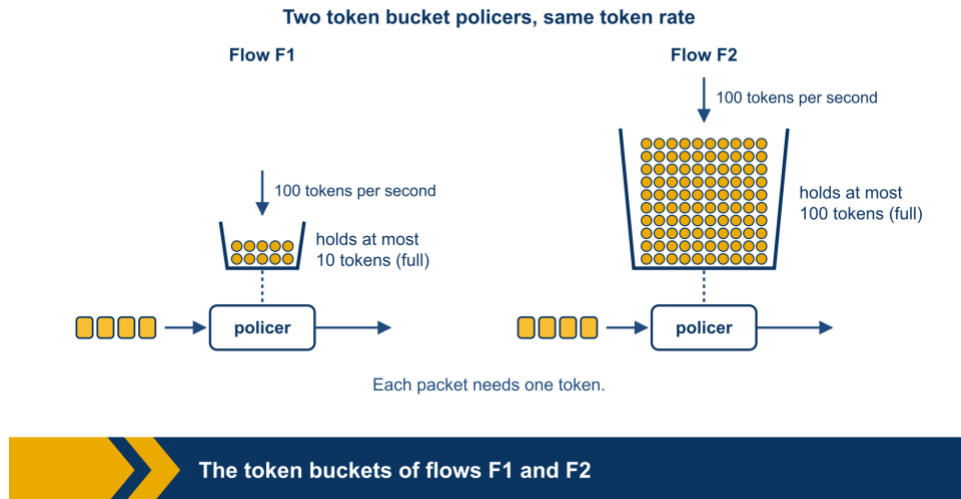


Figure: The token buckets of flows F1 and F2 (Module 6)

Consider a router that polices flows F1 and F2, each with its own token bucket. Both buckets receive $r = 100$ tokens per second. F1's bucket holds at most $b = 10$ tokens, and F2's holds at most $b = 100$. Each packet needs one token.

Both flows have been idle, so both buckets are full.

How do the traffic agreements for F1 and F2 differ?

- **A.** F2 may keep a higher long-run average, while both flows are held to the same largest burst.
- **B.** F2 may send a larger burst at once, while both are held to the same long-run average.
- **C.** F2 may hold up to 100 excess packets in its bucket until they conform, and F1 only 10.
- **D.** F2 may send a larger burst, and its larger bucket also refills with tokens at a faster rate.

Correct answer: **B**

Why: The token rate r sets the long-run average, 100 packets per second for both flows. The bucket size b sets the largest burst after an idle period: 10 packets for F1 and 100 for F2. A token bucket holds tokens, not packets, so a policer drops or marks excess packets instead of queuing them. Both buckets gain tokens at the same rate, so F2 does not refill faster.

Q24. [MCQ]

Consider an ISP that polices the traffic of customer network C with a token bucket at edge router R. Each packet needs one token.

The Service Level Agreement (SLA) allows an average of 1,000 packets per second over the long run, and a burst of up to 200 packets arriving at once after an idle period. The access link carries at most 10,000 packets per second.

Which token bucket settings enforce this SLA?

- **A.** $r = 200$ tokens per second, $b = 1,000$ tokens
- **B.** $r = 10,000$ tokens per second, $b = 200$ tokens

- C. $r = 1,000$ tokens per second, $b = 1,000$ tokens
- D. $r = 1,000$ tokens per second, $b = 200$ tokens

Correct answer: **D**

Why: The token rate sets the long-run average, so $r = 1,000$ tokens per second. The bucket size sets the largest burst, so $b = 200$ tokens. Swapping them allows an average of 200 and bursts of 1,000. A rate of 10,000 matches the link, so it enforces no average. A bucket of 1,000 allows bursts of 1,000, not 200.

Traffic Scheduling: Leaky Bucket

Q25. [MCQ]

Consider a router that shapes a flow with a leaky bucket. The bucket has a fixed-size buffer and drains at a constant rate r .

A long burst arrives much faster than r and fills the buffer.

What happens to packets that arrive while the buffer is full?

- A. They are marked as excess traffic and forwarded immediately.
- B. They wait outside the buffer in an overflow queue, preventing packet loss.
- C. They are transmitted immediately at the burst rate because the bucket is full.
- D. They are dropped, while the bucket keeps sending its queued packets at rate r .

Correct answer: **D**

Why: A leaky bucket shaper buffers arriving packets and leaks them out at a constant rate r . If traffic above r fills the buffer, further packets that arrive at the full bucket are dropped. The bucket is the only buffer, and it never sends faster than r . Marking and forwarding excess packets is what a token bucket policer does.

Q26. [TF]

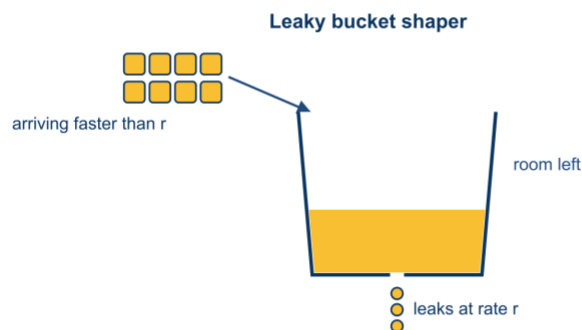


Figure: A leaky bucket with room left (Module 6)

Consider a router that shapes a flow with a leaky bucket that drains at rate r . The flow starts sending faster than r , but the bucket still has room.

The router drops the packets that exceed rate r as soon as they arrive.

- True
- False

Correct answer: **False**

Why: Unlike a policer, a leaky bucket shaper does not drop packets arriving above rate r while buffer space remains. The shaper queues them and releases them at rate r . The shaper drops packets only when the buffer is full.

Q27. [MCQ]

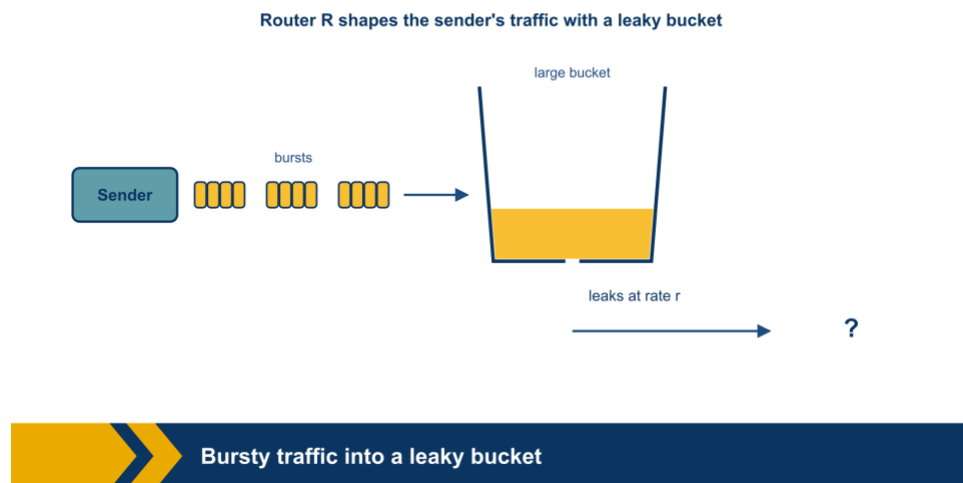


Figure: Bursty traffic into a leaky bucket (Module 6)

Consider a sender S transmitting bursty traffic to router R , which shapes traffic using a leaky bucket shaper that drains at rate r .

The long-run average transmission rate from sender S is less than r , and the leaky bucket buffer is large enough that it never overflows.

What traffic profile leaves router R ?

- A. The bursts pass through unchanged as they arrive, because the average rate is below r .
- B. The bursts leave intact, each burst delayed as a whole until the bucket has drained.
- C. A smoothed stream no faster than r , with burst packets held in the bucket, sent later.
- D. A smoothed stream no faster than r , with every packet that arrives above rate r dropped.

Correct answer: **C**

Why: A leaky bucket shaper removes bursts. The shaper buffers burst arrivals and releases them at a steady rate no greater than r . The buffer never overflows, so no packets are dropped.

Q28. [MCQ]

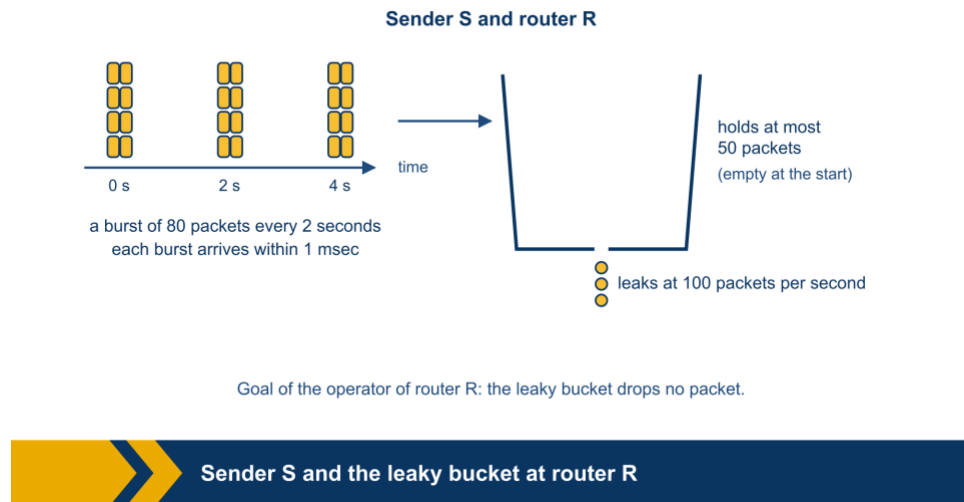


Figure: Sender S and the leaky bucket at router R (Module 6)

Consider a sender S that sends bursty traffic to router R. Every 2 seconds, a burst of 80 packets arrives within 1 millisecond.

Router R shapes this traffic with a leaky bucket that holds at most 50 packets and leaks 100 packets per second, one every 10 ms. The bucket is empty when the first burst arrives. Each burst is so short that no packet leaves the bucket while it arrives.

The operator of router R wants the bucket to drop no packets. Does this leaky bucket meet that goal?

- **A.** No. The 50-packet bucket overflows during each burst, so 30 packets of each burst are dropped.
- **B.** Yes. The average rate of 40 packets per second is well below the leak rate of 100 packets per second.
- **C.** Yes. Excess packets above the leak rate wait in the bucket, so no packets are lost.
- **D.** No. The leaky bucket releases the entire burst at once, exceeding the physical link capacity.

Correct answer: **A**

Why: The 80-packet burst arrives in 1 ms, and no packet leaves during the burst. The first 50 packets fill the bucket, and the other 30 are dropped. The bucket drains in 0.5 seconds, so every burst meets an empty bucket and loses 30 packets. An average below the leak rate prevents loss only when the bucket holds a whole burst. The bucket sends at its leak rate, so no burst leaves at once.